```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style='whitegrid', palette='muted')
plt.rcParams['figure.figsize'] = (10, 5)
pd.set_option('display.max_columns', None)
print("Toolkit ready 👷")
```

```
Toolkit ready 👷
```

```
df = sns.load_dataset('mpg')      # same as UCI Auto-MPG: 398 cars, 1970–1982
print("Shape:", df.shape, "→ 398 cars (rows), 9 facts about each (columns)")
df.head()
```

```
Shape: (398, 9) → 398 cars (rows), 9 facts about each (columns)
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year	origin	name
0	18.0	8	307.0	130.0	3504	12.0	70	usa	chevrolet chevelle malibu
1	15.0	8	350.0	165.0	3693	11.5	70	usa	buick skylark 320
2	18.0	8	318.0	150.0	3436	11.0	70	usa	plymouth satellite
3	16.0	8	304.0	150.0	3433	12.0	70	usa	amc rebel sst
4	17.0	8	302.0	140.0	3449	10.5	70	usa	ford torino

```
df.info()
# Read the 'Non-Null Count' column: anything below 398 is MISSING data.
# Notice: horsepower shows fewer than 398 → some values are missing. Suspicious! 🚩
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mpg           398 non-null    float64
 1   cylinders     398 non-null    int64
 2   displacement  398 non-null    float64
 3   horsepower    392 non-null    float64
```

```
 4   weight        398 non-null    int64
 5   acceleration  398 non-null    float64
 6   model_year    398 non-null    int64
 7   origin        398 non-null    object
 8   name          398 non-null    object
dtypes: float64(4), int64(3), object(2)
memory usage: 28.1+ KB
```

```
df.describe()
# Each column's min, max, mean, and spread in one table.
# Quick reads: mpg ranges ~9 to ~47. Weight ranges from ~1600 to ~5100 lbs — huge range!
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model_year
count	398.000000	398.000000	398.000000	392.000000	398.000000	398.000000	398.000000
mean	23.514573	5.454774	193.425879	104.469388	2970.424623	15.568090	76.010050
std	7.815984	1.701004	104.269838	38.491160	846.841774	2.757689	3.697627
min	9.000000	3.000000	68.000000	46.000000	1613.000000	8.000000	70.000000
25%	17.500000	4.000000	104.250000	75.000000	2223.750000	13.825000	73.000000
50%	23.000000	4.000000	148.500000	93.500000	2803.500000	15.500000	76.000000
75%	29.000000	8.000000	262.000000	126.000000	3608.000000	17.175000	79.000000
max	46.600000	8.000000	455.000000	230.000000	5140.000000	24.800000	82.000000

```
# Missing values — the gaps in our evidence

print("Missing values per column:")
print(df.isnull().sum())

missing_rows = df.isnull().any(axis=1).sum()
print(f"\n{missing_rows} rows have a gap — that's {missing_rows/len(df)*100:.1f}% of our data.")
```

```
Missing values per column:
mpg             0
cylinders       0
displacement    0
horsepower      6
weight          0
acceleration    0
```

```
model_year      0
origin          0
name            0
dtype: int64

6 rows have a gap — that's 1.5% of our data.
```

```
df = df.dropna().reset_index(drop=True)
print("After dropping gaps:", df.shape, "→ clean!")
```

```
After dropping gaps: (392, 9) → clean!
```

```
dupes = df.duplicated().sum()
print("Duplicate rows:", dupes)
df = df.drop_duplicates()
# (This dataset has none — but you ALWAYS check. Duplicates secretly double-count and skew everything.)
```

```
Duplicate rows: 0
```

```
df['cylinders']  = df['cylinders'].astype('category')
df['model_year'] = df['model_year'].astype('category')
df['name']       = df['name'].str.strip().str.lower()   # tidy the text

# Organize our columns into two buckets — we'll analyze each bucket differently
cat_cols = ['cylinders', 'origin', 'model_year']                      # categories (groups)
num_cols = ['mpg', 'displacement', 'horsepower', 'weight', 'acceleration']  # numbers (measurements)
print("🟫 Categorical (groups):", cat_cols)
print("📏 Numerical (measurements):", num_cols)
```

```
🟫 Categorical (groups): ['cylinders', 'origin', 'model_year']
📏 Numerical (measurements): ['mpg', 'displacement', 'horsepower', 'weight', 'acceleration']
```